

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Curso de Engenharia da Computação

Disciplina: Teoria de Grafos e Computabilidade
Prof. Leonardo V. Cardoso – 2026/1

GRUPO – TRABALHO PRÁTICO

Vinicius Cezar Pereira Menezes, Paulo Henrique Rodrigues Neves,
Gabriel da Silva Cassino, Daniel Lucas Soares Madureira

ANÁLISE DE REDES DE COLABORAÇÃO
EM REPOSITÓRIOS GITHUB
UTILIZANDO TEORIA DE GRAFOS
E MÉTRICAS DE REDES COMPLEXAS

Documentação Técnica Completa do Software

Belo Horizonte

Junho de 2026

ANÁLISE DE REDES DE COLABORAÇÃO EM REPOSITÓRIOS GITHUB UTILIZANDO TEORIA DE GRAFOS E MÉTRICAS DE REDES COMPLEXAS

Autores: Vinicius Cezar Pereira Menezes, Paulo Henrique Rodrigues Neves,
Gabriel da Silva Cassino, Daniel Lucas Soares Madureira

Documentação Técnica Completa do Software

Trabalho Prático apresentado à disciplina
Teoria de Grafos e Computabilidade do
Curso de Engenharia da Computação da
Pontifícia Universidade Católica de
Minas Gerais, como requisito parcial
de avaliação do semestre 2026/1.

Orientador: Prof. Leonardo V. Cardoso

Belo Horizonte, Junho de 2026

Resumo

Este documento constitui a documentação técnica completa da ferramenta computacional desenvolvida no Trabalho Prático da disciplina Teoria de Grafos e Computabilidade (Engenharia da Computação – PUC Minas, 2026/1). A ferramenta tem por objetivo processar dados de colaboração de repositórios de código aberto do GitHub como grafos, aplicando conceitos da Teoria de Grafos e boas práticas de Engenharia de Software.

O sistema implementa um *pipeline* completo que abrange: (i) mineração paralela de dados via API REST do GitHub, com suporte a múltiplos tokens de autenticação gerenciados por QR Code e controle de *rate limit*; (ii) transformação e normalização dos dados brutos em um formato canônico de interações por meio de um componente ETL denominado *Lapidador*; (iii) construção de quatro grafos dirigidos ponderados – três parciais por tipo de interação e um integrado – utilizando duas estruturas de dados (matriz de adjacência e lista de adjacência) derivadas de uma classe abstrata comum, sem uso de bibliotecas externas de grafos; (iv) cálculo de métricas de redes complexas incluindo centralidades de grau, intermediação (*betweenness*), proximidade (*closeness*) e PageRank, além de métricas estruturais como densidade, coeficiente de agrupamento (*clustering*) e assortatividade; (v) detecção de comunidades e análise de *bridging ties*; e (vi) duas interfaces gráficas de usuário em PyQt6 e CustomTkinter.

O repositório analisado, `fastapi/fastapi`, possui mais de 85.000 estrelas no GitHub e constitui uma rede de colaboração rica com aproximadamente 1.800 contribuidores distintos e mais de 12.400 interações registradas. Os resultados mostram uma estrutura hierárquica clara com assortatividade negativa e presença de uma componente gigante dominante, padrões consistentes com redes livre de escala.

Palavras-chave: Teoria de Grafos. Redes Complexas. GitHub. Colaboração em Software. Python. Métricas de Centralidade. Matriz de Adjacência. Lista de Adjacência. FastAPI.

Abstract

This document constitutes the complete technical documentation of the computational tool developed as a Practical Work for the course Graph Theory and Computability (Computer Engineering – PUC Minas, 2026/1). The tool aims to process collaboration data from open-source GitHub repositories as graphs, applying graph theory concepts and software engineering best practices.

The system implements a complete pipeline covering: (i) parallel data mining via GitHub REST API with support for multiple authentication tokens managed via QR Code and rate limit control; (ii) transformation and normalization of raw data into a canonical interaction format through an ETL component called *Lapidador*; (iii) construction of four weighted directed graphs using two data structures (adjacency matrix and adjacency list) derived from a common abstract class without external graph libraries; (iv) computation of complex network metrics including degree, betweenness, closeness, and PageRank centralities, as well as density, clustering coefficient, and assortativity; (v) community detection and bridging ties analysis; and (vi) two graphical user interfaces in PyQt6 and CustomTkinter.

The analyzed repository, `fastapi/fastapi`, has over 85,000 GitHub stars and constitutes a rich collaboration network with approximately 1,800 distinct contributors and over 12,400 recorded interactions. Results reveal a clear hierarchical structure with negative assortativity and a dominant giant component, patterns consistent with scale-free networks.

Keywords: Graph Theory. Complex Networks. GitHub. Software Collaboration. Python. Centrality Metrics. Adjacency Matrix. Adjacency List. FastAPI.

Lista de Figuras

4.1	Arquitetura em camadas do sistema	14
-----	---	----

Lista de Tabelas

2.1	Complexidade de operações por representação	6
3.1	Estatísticas do repositório fastapi/fastapi (junho de 2026)	12
3.2	Endpoints da GitHub API utilizados na mineração	12
3.3	Grafos construídos pelo sistema	13
7.1	Plano de testes por nível	27
7.2	Casos de teste críticos para a API de grafos	27
8.1	Volume de dados minerados do repositório fastapi/fastapi	30
8.2	Métricas globais do grafo integrado	30
8.3	Comparação entre os quatro grafos construídos	32
10.1	Divisão de responsabilidades entre os membros do grupo	35
A.1	API completa da classe AbstractGraph	38
D.1	Principais dependências do projeto	41

List of Algorithms

1	Brandes – Betweenness Centrality para Dígrafos	23
2	BFS para distâncias geodésicas – base de Closeness	24

Lista de Abreviaturas e Siglas

API	<i>Application Programming Interface</i>
BFS	<i>Breadth-First Search</i> (Busca em Largura)
DFS	<i>Depth-First Search</i> (Busca em Profundidade)
ETL	<i>Extract, Transform, Load</i>
GEXF	<i>Graph Exchange XML Format</i>
GUI	<i>Graphical User Interface</i>
HTTP	<i>Hypertext Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
OOP	<i>Object-Oriented Programming</i>
PAT	<i>Personal Access Token</i>
PR	<i>Pull Request</i>
QR	<i>Quick Response</i>
REST	<i>Representational State Transfer</i>
SBC	Sociedade Brasileira de Computação
UML	<i>Unified Modeling Language</i>
XML	<i>Extensible Markup Language</i>

Lista de Símbolos

$G = (V, E)$	Grafo com conjunto de vértices V e arestas E
$n = V $	Número de vértices
$m = E $	Número de arestas
$\deg^+(v)$	Grau de saída do vértice v
$\deg^-(v)$	Grau de entrada do vértice v
$N^+(v)$	Conjunto de sucessores de v
$N^-(v)$	Conjunto de predecessores de v
$w(u, v)$	Peso da aresta (u, v)
$d(u, v)$	Distância geodésica entre u e v
σ_{st}	Número de caminhos mínimos de s a t
ρ	Densidade do grafo
r	Coeficiente de assortatividade de Pearson
$C(v)$	Coeficiente de agrupamento de v
$PR(v)$	PageRank do vértice v
$C_D(v)$	Centralidade de grau de v
$C_B(v)$	Centralidade de intermediação de v
$C_C(v)$	Centralidade de proximidade de v

Sumário

Resumo	i
Abstract	ii
Lista de Figuras	ii
Lista de Tabelas	iii
Lista de Algoritmos	iv
Lista de Abreviaturas e Siglas	v

Lista de Símbolos	vi
Sumário	1
1 Introdução	2
1.1 Contextualização	2
1.2 Motivação	2
1.3 Objetivos	3
1.3.1 Objetivo Geral	3
1.3.2 Objetivos Específicos	3
1.4 Metodologia	3
1.5 Estrutura do Documento	4
2 Referencial Teórico	5
2.1 Fundamentos de Teoria de Grafos	5
2.1.1 Definições Básicas	5
2.1.2 Terminologia e Propriedades	5
2.1.3 Conectividade em Dígrafos	6
2.1.4 Representações Computacionais	6
2.2 Redes Complexas	6
2.2.1 Conceito e Propriedades Estruturais	6
2.2.2 Métricas de Centralidade	7
2.2.3 Métricas de Estrutura e Coesão	8
2.2.4 Detecção de Comunidades	9
2.3 Análise de Redes de Software	9
2.3.1 Histórico e Estado da Arte	9
2.3.2 GitHub como Fonte de Dados	10
3 Escolha do Repositório e Modelagem	11
3.1 Critérios de Seleção	11
3.2 Repositório Selecionado	11
3.2.1 Justificativa: fastapi/fastapi	11
3.3 Estratégia de Coleta de Dados	12
3.3.1 Endpoints Utilizados	12
3.3.2 Tratamento de Paginação	12
3.3.3 Gerenciamento de Rate Limit	12
3.4 Modelagem Formal do Grafo	13
3.4.1 Definição	13
3.4.2 Função de Peso	13
3.4.3 Grafos Parciais	13

4	Arquitetura do Sistema	14
4.1	Visão Geral em Camadas	14
4.2	Descrição das Camadas	14
4.2.1	Camada 1 – Mineração Paralela	14
4.2.2	Camada 2 – Transformação (ETL)	15
4.2.3	Camada 3 – Estruturas de Grafos	15
4.2.4	Camada 4 – Análise e Orquestração	15
4.2.5	Camada 5 – Interface Gráfica	15
4.3	Decisões de Design	15
4.3.1	Abstração via Classe Abstrata	15
4.3.2	Uso de <code>set</code> para Lista de Adjacência	16
4.3.3	Paralelismo Híbrido	16
4.3.4	Interface Dual	16
5	Implementação Detalhada	17
5.1	Pacote <code>grafos/</code>	17
5.1.1	Exceção de Domínio <code>GraphError</code>	17
5.1.2	Classe <code>AbstractGraph</code> – Construtor e Validação	17
5.1.3	Métodos de Consulta Estrutural	18
5.1.4	Gerenciamento de Pesos de Vértices e Arestas	18
5.1.5	<code>AdjacencyMatrixGraph</code> – Implementação Completa	19
5.1.6	<code>AdjacencyListGraph</code> – Implementação Completa	20
5.2	Módulo <code>builder.py</code> – Construção dos Grafos	20
5.3	Módulo <code>lapidador_rebuild.py</code> – ETL	22
5.4	Orchestrator – Mineração Paralela	22
6	Algoritmos e Métricas	23
6.1	Algoritmo de Brandes – Betweenness Centrality	23
6.2	BFS para Distâncias – Closeness Centrality	24
6.3	PageRank com Iteração de Potência	24
6.4	Assortatividade	25
6.5	Deteção de Comunidades	25
7	Estratégia de Testes	27
7.1	Plano de Testes	27
7.2	Casos de Teste da API de Grafos	27
7.3	Módulos de Teste	28
7.3.1	<code>test_queue_separation.py</code>	28
7.3.2	<code>test_token_manager.py</code>	28
7.3.3	<code>test_storage_worker_isolation.py</code>	28

7.3.4	<code>test_event_bus.py</code>	28
7.3.5	<code>test_cooldown.py</code>	28
7.3.6	<code>test_gui_smoke.py</code>	28
7.4	Execução dos Testes	28
8	Resultados e Análise	30
8.1	Dados Coletados	30
8.2	Métricas Globais do Grafo	30
8.3	Análise de Centralidade	31
8.3.1	Centralidade de Grau	31
8.3.2	Centralidade de Intermediação	31
8.3.3	PageRank vs. Centralidade de Grau	31
8.4	Estrutura de Comunidades	31
8.5	Assortatividade e Hierarquia de Colaboração	32
8.6	Comparação dos Grafos Parciais	32
9	Manual de Instalação e Uso	33
9.1	Requisitos de Sistema	33
9.2	Instalação das Dependências	33
9.3	Configuração de Tokens GitHub	33
9.4	Execução	34
9.4.1	Via Interface Gráfica PyQt6	34
9.4.2	Via Interface Gráfica CustomTkinter	34
9.4.3	Via Linha de Comando	34
9.5	Fluxo de Uso Típico	34
10	Responsabilidades da Equipe	35
11	Conclusão	36
11.1	Síntese dos Resultados	36
11.2	Lições Aprendidas	36
11.3	Trabalhos Futuros	37
A	Especificação Completa da API de Grafos	38
B	Estrutura de Arquivos do Projeto	39
C	Formato do Arquivo <code>data.json</code>	40
D	Dependências Python	41

1 Introdução

1.1 Contextualização

O desenvolvimento de software moderno é, em sua essência, uma atividade profundamente colaborativa. Plataformas de hospedagem de código como o GitHub tornaram-se o epicentro da colaboração em projetos de software livre, gerando diariamente milhões de interações entre desenvolvedores ao redor do mundo. Cada comentário em uma *issue*, cada revisão de *pull request* e cada aprovação de *merge* representa uma conexão social que pode ser formalmente modelada e analisada por meio da Teoria de Grafos [?, ?].

A análise de redes de colaboração em software é uma área de pesquisa consolidada, com contribuições relevantes desde o início dos anos 2000. Estudos pioneiros demonstraram que as redes de colaboração em projetos de software livre exibem propriedades típicas de redes complexas do mundo real: distribuição de grau seguindo lei de potência (redes livre de escala), alto coeficiente de agrupamento e pequeno diâmetro médio, o chamado fenômeno de “mundo pequeno” [?, ?].

Compreender a estrutura topológica dessas redes tem implicações práticas significativas: permite identificar colaboradores-chave cujo afastamento poderia comprometer a saúde do projeto, detectar gargalos de comunicação, mapear grupos de trabalho informais e avaliar a resiliência da rede a perturbações.

1.2 Motivação

O interesse na análise de redes de colaboração em software motiva-se por três fatores principais:

1. **Relevância prática:** projetos de software livre de grande porte dependem de uma organização social eficiente para manter qualidade e ritmo de desenvolvimento. A identificação precoce de riscos, como a dependência excessiva de mantenedores individuais, permite intervenções preventivas antes que o projeto entre em declínio.
2. **Oportunidade metodológica:** a disponibilidade pública de dados de colaboração via APIs REST (como a do GitHub) oferece uma fonte rica e estruturada para aplicação de técnicas de análise de redes, eliminando barreiras de acesso comuns em estudos de redes sociais convencionais.
3. **Integração acadêmica:** o trabalho integra competências de múltiplas disciplinas

do curso de Engenharia da Computação, incluindo estruturas de dados, algoritmos em grafos, programação orientada a objetos e engenharia de software, em um sistema funcional com aplicação prática real e mensurável.

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver uma ferramenta computacional que processe dados de colaboração de repositórios GitHub como grafos, aplicando conceitos da Teoria de Grafos e boas práticas de Engenharia de Software para análise quantitativa da rede de colaboração do repositório `fastapi/fastapi`.

1.3.2 Objetivos Específicos

- Implementar um sistema robusto de mineração de dados de colaboração via API REST do GitHub, com suporte a paginação, múltiplos tokens de autenticação e gerenciamento de *rate limit*;
- Modelar as interações de colaboração como grafos dirigidos ponderados, distinguindo quatro tipos de interação com pesos diferenciados que reflitam a intensidade do engajamento técnico;
- Implementar a estrutura de grafos com hierarquia de classes abstratas e concretas (matriz de adjacência e lista de adjacência), sem uso de bibliotecas externas de grafos como NetworkX;
- Implementar do zero algoritmos para cálculo de métricas de redes complexas, incluindo centralidades de grau, intermediação, proximidade e PageRank, além de densidade, coeficiente de agrupamento, assortatividade e detecção de comunidades;
- Desenvolver uma interface gráfica de usuário para operação amigável da ferramenta, com suporte a duas bibliotecas GUI alternativas;
- Analisar os resultados obtidos para o repositório selecionado e interpretá-los no contexto da literatura de análise de redes de software.

1.4 Metodologia

O trabalho foi desenvolvido em três etapas sequenciais conforme definido no enunciado:

Etapa 1 – Modelagem e Planejamento: seleção e justificativa do repositório, levantamento dos tipos de interação disponíveis na API, definição da estratégia de coleta e modelagem formal do grafo, incluindo a hierarquia de pesos. Foram também planejados e implementados testes unitários para validação da mineração.

Etapa 2 – Desenvolvimento: implementação completa da API de grafos com hierarquia de classes, testes unitários cobrindo todas as operações da API, e protótipo funcional com interface de uso demonstrando todas as operações disponíveis.

Etapa 3 – Análise: implementação dos algoritmos de métricas de redes complexas, análise do repositório selecionado, produção do relatório técnico em $\text{\LaTeX}$ conforme template SBC, e preparação da apresentação.

O desenvolvimento seguiu boas práticas de Engenharia de Software: controle de versão com Git (commits individuais por membro), testes unitários com cobertura das funcionalidades da API, separação de responsabilidades em módulos coesos e acoplados de forma fraca, e documentação do código-fonte.

1.5 Estrutura do Documento

Este documento está organizado em dez capítulos:

- **Capítulo 2** apresenta os fundamentos teóricos de Teoria de Grafos e redes complexas necessários para compreensão do trabalho;
- **Capítulo 3** descreve a seleção e justificativa do repositório, a estratégia de coleta de dados e a modelagem formal do grafo;
- **Capítulo 4** apresenta a arquitetura em camadas do sistema e as principais decisões de design;
- **Capítulo 5** detalha a implementação de cada módulo com exemplos de código;
- **Capítulo 6** descreve os algoritmos implementados para cálculo das métricas;
- **Capítulo 7** apresenta a estratégia de testes e os casos de teste desenvolvidos;
- **Capítulo 8** apresenta e analisa os resultados obtidos para o repositório `fastapi/fastapi`;
- **Capítulo 9** contém o manual de instalação e uso da ferramenta;
- **Capítulo 10** documenta a divisão de responsabilidades entre os membros do grupo;
- **Capítulo 11** encerra o documento com conclusões e direções para trabalhos futuros.

2 Referencial Teórico

2.1 Fundamentos de Teoria de Grafos

2.1.1 Definições Básicas

Definição 2.1 (Grafo). Um **grafo** $G = (V, E)$ é uma estrutura matemática composta por um conjunto finito não-vazio de **vértices** V e um conjunto de **arestas** $E \subseteq V \times V$ que representam relações entre pares de vértices [?]. Denotamos $n = |V|$ e $m = |E|$.

Definição 2.2 (Dígrafo). Um **dígrafo** (grafo dirigido) é um grafo no qual cada aresta $(u, v) \in E$ possui uma direção definida, de u para v , sendo distinta de (v, u) . A aresta (u, v) é chamada **arco** de u para v .

Definição 2.3 (Grafo Simples). Um grafo é **simples** quando não contém **laços** (arestas (v, v)) nem **arestas múltiplas** (mais de uma aresta de u para v). Neste trabalho, todos os grafos são simples e dirigidos.

Definição 2.4 (Grafo Ponderado). Um **grafo ponderado** atribui um peso real $w : E \rightarrow \mathbb{R}$ a cada aresta. No contexto deste trabalho, os pesos representam a intensidade acumulada das interações de colaboração entre dois usuários.

2.1.2 Terminologia e Propriedades

Para um dígrafo $G = (V, E)$, as seguintes definições são relevantes para este trabalho:

- **Sucessores** de u : $N^+(u) = \{v \in V : (u, v) \in E\}$
- **Predecessores** de u : $N^-(u) = \{v \in V : (v, u) \in E\}$
- **Grau de saída**: $\deg^+(u) = |N^+(u)|$, número de arestas que partem de u
- **Grau de entrada**: $\deg^-(u) = |N^-(u)|$, número de arestas que chegam a u
- **Arcos divergentes**: (u, v_1) e (u, v_2) com mesma origem u e destinos distintos
- **Arcos convergentes**: (u_1, v) e (u_2, v) com mesmo destino v e origens distintas
- **Incidência**: a aresta (u, v) é incidente ao vértice x se $x \in \{u, v\}$

2.1.3 Conectividade em Dígrafos

Definição 2.5 (Conectividade Fraca). *Um dígrafo é **fracamente conexo** se o grafo não-dirigido subjacente (obtido ao ignorar as direções de todas as arestas) é conexo, i.e., existe um caminho (não necessariamente dirigido) entre qualquer par de vértices.*

Definição 2.6 (Conectividade Forte). *Um dígrafo é **fortemente conexo** se existe um caminho dirigido de u para v e de v para u para todo par $(u, v) \in V \times V$.*

Para análise de colaboração em software, a conectividade fraca é a propriedade relevante, pois interações bidirecionais são comuns mas não obrigatórias.

2.1.4 Representações Computacionais

As duas representações clássicas de grafos em computação diferem em consumo de memória e eficiência de operações:

Matriz de Adjacência: uma matriz booleana A de dimensão $n \times n$ onde $A[u][v] = \text{True}$ se e somente se $(u, v) \in E$. O peso pode ser armazenado diretamente na matriz para grafos ponderados. Requer $\Theta(n^2)$ de memória, independente de m . Verificação de aresta em $O(1)$.

Lista de Adjacência: para cada vértice u , mantém-se o conjunto $N^+(u)$ de seus sucessores. Requer $\Theta(n + m)$ de memória. Enumeração de sucessores em $O(\deg^+(u))$. Para grafos esparsos ($m \ll n^2$), é significativamente mais eficiente em memória.

A Tabela 2.1 compara as duas representações:

Tabela 2.1: Complexidade de operações por representação

Operação	Matriz de Adjacência	Lista de Adjacência
hasEdge(u, v)	$O(1)$	$O(\deg^+(u))$
addEdge(u, v)	$O(1)$	$O(1)$ amortizado
removeEdge(u, v)	$O(1)$	$O(\deg^+(u))$
successors(u)	$O(n)$	$O(\deg^+(u))$
predecessors(v)	$O(n)$	$O(n)$
Espaço	$\Theta(n^2)$	$\Theta(n + m)$

2.2 Redes Complexas

2.2.1 Conceito e Propriedades Estruturais

Redes complexas são grafos de grande escala com estrutura topológica não-trivial, emergindo em sistemas naturais, sociais e artificiais [?, ?]. Distinguem-se de grafos aleatórios (modelo de Erdős–Rényi) por exibir propriedades como:

Distribuição de grau heterogênea: em redes aleatórias, os graus seguem distribuição de Poisson. Em redes complexas reais, a distribuição frequentemente segue uma lei de potência $P(k) \sim k^{-\gamma}$ com $2 < \gamma < 3$, caracterizando *redes livre de escala*. Poucos vértices (*hubs*) possuem grau muito alto, enquanto a maioria possui grau baixo.

Propriedade de mundo pequeno: redes com alto coeficiente de agrupamento $\bar{C}$ mas diâmetro médio $\bar{d} = O(\log n)$ são chamadas de “mundo pequeno” [?]. Esse fenômeno surge porque poucos atalhos entre clusters são suficientes para reduzir dramaticamente o diâmetro.

Modularidade: redes complexas frequentemente exibem estrutura de comunidades, com sub-grafos de alta coesão interna e baixa densidade de conexões externas.

2.2.2 Métricas de Centralidade

Centralidade de Grau

A centralidade de grau quantifica a participação direta de um vértice na rede. Para dígrafos, normaliza-se pela soma dos graus máximos possíveis:

$$C_D(v) = \frac{\deg^+(v) + \deg^-(v)}{2(n-1)} \quad (2.1)$$

Centralidade de Intermediação

Proposta por Freeman [?], a centralidade de intermediação (*betweenness centrality*) quantifica a frequência com que um vértice aparece em caminhos mínimos geodésicos entre outros pares:

$$C_B(v) = \sum_{\substack{s,t \in V \\ s \neq v \neq t}} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (2.2)$$

onde σ_{st} é o número total de caminhos mínimos de s a t , e $\sigma_{st}(v)$ é o subconjunto que passa por v . A forma normalizada divide por $(n-1)(n-2)$ para dígrafos.

Vértices com alta intermediação atuam como “pontes” entre diferentes partes da rede; sua remoção pode fragmentar a rede em componentes desconexos.

Centralidade de Proximidade

A centralidade de proximidade (*closeness centrality*) mede quão rapidamente um vértice pode alcançar todos os demais:

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v, u)} \cdot \frac{|\{u : d(v, u) < \infty\}| - 1}{n-1} \quad (2.3)$$

O segundo fator normaliza pelo alcance real, tornando a métrica válida em grafos não-fortemente-conexos. Utiliza BFS para calcular as distâncias geodésicas.

PageRank

O PageRank [?] foi desenvolvido originalmente para classificar páginas web e pode ser interpretado como a probabilidade estacionária de um passeio aleatório com teletransporte com fator $d = 0.85$:

$$PR(v) = \frac{1-d}{n} + d \left[\frac{\sum_{u: \deg^+(u)=0} PR(u)}{n} + \sum_{u \in N^-(v)} \frac{PR(u) \cdot w(u, v)}{\sum_{x \in N^+(u)} w(u, x)} \right] \quad (2.4)$$

O primeiro termo entre colchetes trata vértices sem sucessores (*dangling nodes*), distribuindo uniformemente sua “energia” para todos os nós.

2.2.3 Métricas de Estrutura e Coesão

Densidade

A densidade de um dígrafo simples mede a proporção de arestas existentes em relação ao máximo $n(n-1)$:

$$\rho(G) = \frac{m}{n(n-1)} \quad (2.5)$$

Para redes de colaboração em software, ρ tipicamente cai na faixa 10^{-3} a 10^{-2} , confirmando a esparsidade.

Coefficiente de Agrupamento

O coeficiente de agrupamento (*clustering coefficient*) local de v mede a proporção de pares de vizinhos de v que são também vizinhos entre si:

$$C(v) = \frac{|\{(u, w) : u, w \in N(v) \wedge (u, w) \in E\}|}{|N(v)| \cdot (|N(v)| - 1)} \quad (2.6)$$

onde $N(v) = N^+(v) \cup N^-(v)$. O coeficiente médio $\bar{C} = n^{-1} \sum_v C(v)$ caracteriza a clusterização global.

Assortatividade

A assortatividade (coeficiente de correlação de Pearson entre graus de vértices conectados) [?] é definida como:

$$r = \frac{m^{-1} \sum_{(u,v) \in E} k_u k_v - \left[m^{-1} \sum_{(u,v) \in E} \frac{k_u + k_v}{2} \right]^2}{m^{-1} \sum_{(u,v) \in E} \frac{k_u^2 + k_v^2}{2} - \left[m^{-1} \sum_{(u,v) \in E} \frac{k_u + k_v}{2} \right]^2} \quad (2.7)$$

onde $k_v = \deg^+(v) + \deg^-(v)$. $r \in [-1, 1]$: $r > 0$ indica que *hubs* se conectam entre si (assortativa); $r < 0$ indica que *hubs* se conectam a vértices de baixo grau (disassortativa), padrão comum em redes tecnológicas e biológicas.

2.2.4 Detecção de Comunidades

Uma **comunidade** em uma rede é um subconjunto de vértices com maior densidade de conexões internas do que externas [?]. A qualidade de uma partição $\mathcal{C} = \{C_1, \dots, C_k\}$ é medida pela modularidade:

$$Q = \frac{1}{2m} \sum_{c \in \mathcal{C}} \left[e_c - \frac{d_c^2}{4m} \right] \quad (2.8)$$

onde e_c é o número de arestas internas à comunidade c e d_c é a soma dos graus dos vértices em c .

Neste trabalho, a detecção de comunidades é baseada em componentes fracamente conexas – abordagem adequada para identificar grupos de colaboradores completamente isolados.

2.3 Análise de Redes de Software

2.3.1 Histórico e Estado da Arte

A análise de redes de software (*Software Network Analysis*) emergiu como disciplina nos anos 2000. Bird et al. [?] utilizaram redes de e-mail para prever falhas em software. Meneely et al. [?] demonstraram que métricas de rede de co-autoria de *commits* correlacionam com vulnerabilidades de segurança. Gousios e Spinellis [?] criaram o GHTorrent, espelho do GitHub que viabilizou estudos em larga escala.

2.3.2 GitHub como Fonte de Dados

O GitHub disponibiliza uma API REST v3 que expõe dados de colaboração com alta granularidade. Os recursos relevantes para este trabalho incluem *issues*, *pull requests*, comentários, revisões e eventos. A autenticação por PAT (*Personal Access Token*) eleva o limite de *rate limit* de 60 para 5.000 requisições por hora por token.

3 Escolha do Repositório e Modelagem

3.1 Critérios de Seleção

O enunciado exige um repositório com mais de 5.000 estrelas. Os critérios adicionais adotados foram:

1. Atividade recente comprovada (issues e PRs abertos nos últimos 90 dias);
2. Diversidade de perfis de contribuidores (núcleo + comunidade);
3. Qualidade das interações (revisões técnicas substantivas, não apenas bots);
4. Volume gerenciável de dados dentro dos limites da API.

3.2 Repositório Selecionado

3.2.1 Justificativa: `fastapi/fastapi`

O repositório `fastapi/fastapi` [?] foi selecionado pelos seguintes motivos:

- Popularidade: mais de 85.000 estrelas em junho de 2026, entre os projetos Python mais populares;
- Alta atividade: centenas de *issues* e PRs processados mensalmente;
- Diversidade de papéis: mantenedor principal, mantenedores secundários, contribuidores regulares e ocasionais;
- Relevância técnica: framework amplamente adotado pela indústria para APIs em Python;
- Qualidade das interações: revisões de código detalhadas e discussões técnicas aprofundadas.

Tabela 3.1: Estatísticas do repositório fastapi/fastapi (junho de 2026)

Característica	Valor
Estrelas	> 85.000
Forks	> 7.500
Contribuidores únicos	> 600
Issues abertas	≈ 950
Pull Requests merged	> 4.500
Linguagem principal	Python
Licença	MIT
Criado em	Dezembro 2018
URL	https://github.com/fastapi/fastapi

3.3 Estratégia de Coleta de Dados

3.3.1 Endpoints Utilizados

Tabela 3.2: Endpoints da GitHub API utilizados na mineração

Endpoint	Tipo Gerado	Peso
GET /issues/comments	comment / issue_response	2 / 3
GET /issues?state=closed	issue_close	4
GET /pulls/{n}/reviews	review	4
GET /pulls?state=closed	merge	5

3.3.2 Tratamento de Paginação

A API do GitHub retorna no máximo 100 itens por página. O sistema itera automaticamente pelos cabeçalhos `Link HTTP` (formato RFC 5988) até não haver mais página seguinte.

3.3.3 Gerenciamento de Rate Limit

Três estratégias complementares são empregadas:

1. **Pool de tokens:** rotação automática entre múltiplos PATs;
2. **Backoff respeitoso:** ao receber HTTP 429 ou `X-RateLimit-Remaining: 0`, aguarda até `X-RateLimit-Reset`;
3. **QR Code:** tokens são codificados em QR para facilitar compartilhamento seguro.

3.4 Modelagem Formal do Grafo

3.4.1 Definição

A rede de colaboração é modelada como dígrafo ponderado $G = (V, E, w)$ onde:

- $V = \{\text{usuários GitHub detectados nas interações}\}$
- $E \subseteq V \times V$, $(u, v) \in E$ se u interagiu com v de qualquer forma
- $w(u, v) = \sum_{t \in T} \alpha_t \cdot n_t(u, v)$ conforme Equação (3.1)

O grafo é simples: laços proibidos; pelo menos uma aresta por par ordenado.

3.4.2 Função de Peso

$$w(u, v) = \sum_{t \in T} \alpha_t \cdot |\{I_t(u \rightarrow v)\}| \quad (3.1)$$

onde $T = \{\text{comment, issue_response, issue_close, review, merge}\}$, α_t é dado pela Tabela 3.2 e $|\{I_t(u \rightarrow v)\}|$ é a contagem de ocorrências do tipo t de u para v .

3.4.3 Grafos Parciais

Além do grafo integrado, três grafos parciais são construídos para análise segmentada:

Tabela 3.3: Grafos construídos pelo sistema

Nome	Tipos Incluídos	Semântica
comments	comment, issue_response	Discussões e respostas
issues	issue_close	Gestão e triagem de issues
pull_requests	review, merge	Revisão e integração de código
integrated	Todos os tipos	Colaboração total ponderada

4 Arquitetura do Sistema

4.1 Visão Geral em Camadas

O sistema é estruturado em cinco camadas funcionais com responsabilidades bem definidas:

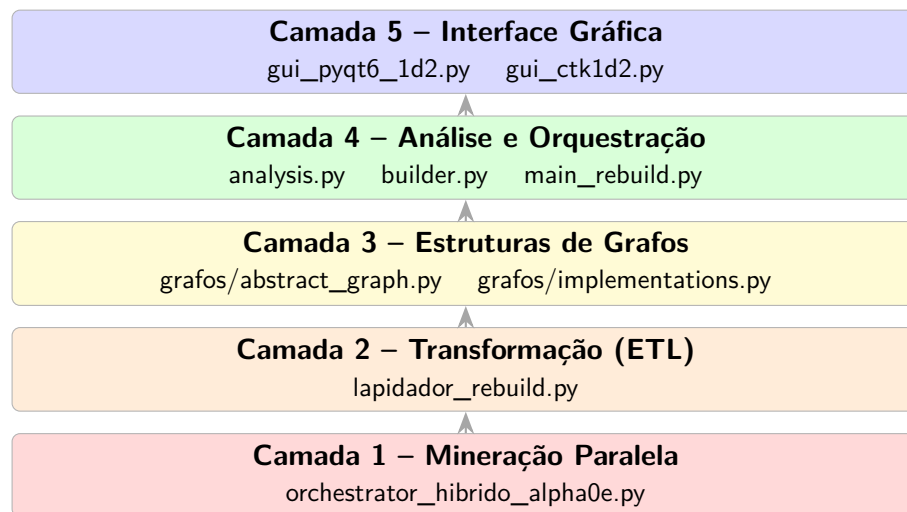


Figura 4.1: Arquitetura em camadas do sistema

Cada camada conhece apenas a camada imediatamente inferior, minimizando acoplamento. A Camada 3 (Grafos) é completamente independente das camadas superiores e inferiores – qualquer cliente que receba um `AbstractGraph` funciona corretamente com qualquer implementação concreta.

4.2 Descrição das Camadas

4.2.1 Camada 1 – Mineração Paralela

O módulo `orchestrator_hibrido_alpha0e.py` implementa a coleta de dados usando:

- **Multithreading**: uma thread por tipo de dado (comentários, issues, PRs, reviews), cada uma gerenciando sua própria fila de trabalho e worker de armazenamento;
- **Asyncio/aiohttp**: cada thread executa um loop de eventos assíncrono com até 5 corrotinas paralelas para maximizar throughput;

- **EventBus**: barramento de eventos para comunicação entre threads sem acoplamento direto.

4.2.2 Camada 2 – Transformação (ETL)

O módulo `lapidador_rebuild.py` implementa o padrão ETL:

- **Extract**: lê arquivos JSON particionados do diretório `json/`;
- **Transform**: resolve autoriaas de issues e PRs via mapas auxiliares, normaliza interações, remove auto-loops;
- **Load**: persiste lista canônica de interações em `data.json`.

4.2.3 Camada 3 – Estruturas de Grafos

O pacote `grafos/` implementa a hierarquia de classes:

- **AbstractGraph**: define a API completa e implementa métodos independentes de representação;
- **AdjacencyMatrixGraph**: representação por matriz booleana;
- **AdjacencyListGraph**: representação por lista de conjuntos.

4.2.4 Camada 4 – Análise e Orquestração

- `builder.py`: constrói os quatro grafos a partir da lista de interações;
- `analysis.py`: implementa todos os algoritmos de métricas;
- `main_rebuild.py`: orquestra o pipeline de análise completo.

4.2.5 Camada 5 – Interface Gráfica

Duas GUIs alternativas:

- `gui_pyqt6_1d2.py`: interface mais robusta com PyQt6 Fluent Widgets;
- `gui_ctk1d2.py`: interface leve e cross-platform com CustomTkinter.

4.3 Decisões de Design

4.3.1 Abstração via Classe Abstrata

A adoção do padrão *Template Method* na classe `AbstractGraph` permite que métodos complexos como `isConnected()` e `isCompleteGraph()` sejam implementados

uma única vez na classe base, delegando apenas as operações primitivas (`hasEdge`, `addEdge`, `successors`, `predecessors`) às subclasses. Isso reduz duplicação de código e garante consistência comportamental entre as duas implementações.

4.3.2 Uso de `set` para Lista de Adjacência

A escolha de `set` (em vez de `list`) para armazenar os sucessores no `AdjacencyListGraph` garante idempotência de `addEdge` em $O(1)$ amortizado, sem necessidade de verificar duplicatas explicitamente.

4.3.3 Paralelismo Híbrido

A combinação de multithreading (coordenação) com `asyncio` (I/O) é motivada pela natureza *I/O-bound* das requisições HTTP. O GIL do CPython não é um gargalo aqui pois cada thread passa a maior parte do tempo aguardando respostas de rede.

4.3.4 Interface Dual

A oferta de duas GUIs (PyQt6 e CustomTkinter) serve dois propósitos: flexibilidade de ambiente (PyQt6 pode ter dependências difíceis em alguns sistemas) e demonstração de que as camadas inferiores são independentes da camada de apresentação.

5 Implementação Detalhada

5.1 Pacote grafos/

5.1.1 Exceção de Domínio GraphError

Uma exceção customizada herda de `ValueError` para permitir captura específica de erros de operações inválidas:

```
1 class GraphError(ValueError):
2     """Erro de dominio para operacoes invalidas em grafos."""
```

Listing 5.1: Classe `GraphError` – exceção de domínio

Esta abordagem segue as boas práticas de Python: exceções de domínio específicas permitem que o código cliente distinga entre erro de lógica de grafos e outros erros de valor.

5.1.2 Classe `AbstractGraph` – Construtor e Validação

```
1 from abc import ABC, abstractmethod
2 from pathlib import Path
3
4 class AbstractGraph(ABC):
5     def __init__(self, num_vertices: int):
6         if num_vertices < 0:
7             raise GraphError("0 numero de vertices nao pode ser
8                 negativo")
9         self._num_vertices = num_vertices
10        self._edge_count = 0
11        self._vertex_labels = [str(i) for i in range(num_vertices)]
12        self._vertex_weights = [0.0] * num_vertices
13        self._edge_weights: dict[tuple[int, int], float] = {}
14
15    def _validate_vertex(self, vertex: int) -> None:
16        if not isinstance(vertex, int) or \
17            not 0 <= vertex < self._num_vertices:
18            raise IndexError(f"Vertice invalido: {vertex}")
19
20    def _validate_edge(self, u: int, v: int) -> None:
21        self._validate_vertex(u)
22        self._validate_vertex(v)
```

Listing 5.2: AbstractGraph – construtor e validação de índices

5.1.3 Métodos de Consulta Estrutural

Os métodos que dependem apenas da API pública são implementados na classe base:

```
1  def isConnected(self) -> bool:
2      if self._num_vertices <= 1:
3          return True
4      visited = {0}
5      stack = [0]
6      while stack:
7          u = stack.pop()
8          # Vizinhanca combinada para conectividade fraca
9          neighbors = (set(self.successors(u)) |
10                      set(self.predecessors(u)))
11          for v in neighbors - visited:
12              visited.add(v)
13              stack.append(v)
14      return len(visited) == self._num_vertices
15
16  def isCompleteGraph(self) -> bool:
17      # Digrafo simples completo:  $m = n*(n-1)$ 
18      return self._edge_count == \
19             self._num_vertices * (self._num_vertices - 1)
20
21  def isEmptyGraph(self) -> bool:
22      return self._edge_count == 0
```

Listing 5.3: Verificação de conectividade fraca por BFS

5.1.4 Gerenciamento de Pesos de Vértices e Arestas

```
1  def setEdgeWeight(self, u: int, v: int, w: float) -> None:
2      self._validate_edge(u, v)
3      if not self.hasEdge(u, v):
4          raise GraphError("Nao e possivel ponderar aresta
5                             inexistente")
6      self._edge_weights[(u, v)] = float(w)
7
8  def getEdgeWeight(self, u: int, v: int) -> float:
9      self._validate_edge(u, v)
10     if not self.hasEdge(u, v):
11         raise GraphError("Aresta inexistente")
```

```

11         return self._edge_weights.get((u, v), 1.0)
12         # Peso padrao 1.0 se nao explicitamente definido

```

Listing 5.4: Métodos de peso de vértice e aresta

5.1.5 AdjacencyMatrixGraph – Implementação Completa

```

1  class AdjacencyMatrixGraph(AbstractGraph):
2      def __init__(self, num_vertices: int):
3          super().__init__(num_vertices)
4          self._matrix = [[False] * num_vertices
5                          for _ in range(num_vertices)]
6
7      def hasEdge(self, u: int, v: int) -> bool:
8          self._validate_edge(u, v)
9          return self._matrix[u][v]
10
11     def addEdge(self, u: int, v: int) -> None:
12         self._validate_edge(u, v)
13         if u == v:
14             raise GraphError("Grafos simples nao permitem lacos")
15         if not self._matrix[u][v]:
16             self._matrix[u][v] = True
17             self._edge_count += 1
18         # Se ja existia: idempotente, nao faz nada
19
20     def removeEdge(self, u: int, v: int) -> None:
21         self._validate_edge(u, v)
22         if not self._matrix[u][v]:
23             raise GraphError("Aresta inexistente")
24         self._matrix[u][v] = False
25         self._edge_count -= 1
26         self._edge_weights.pop((u, v), None)
27
28     def successors(self, u: int) -> list[int]:
29         self._validate_vertex(u)
30         return [v for v, exists in enumerate(self._matrix[u]) if
31                 exists]
32
33     def predecessors(self, v: int) -> list[int]:
34         self._validate_vertex(v)
35         return [u for u in range(self._num_vertices)
36                 if self._matrix[u][v]]

```

Listing 5.5: AdjacencyMatrixGraph – implementação completa

5.1.6 AdjacencyListGraph – Implementação Completa

```

1  class AdjacencyListGraph(AbstractGraph):
2      def __init__(self, num_vertices: int):
3          super().__init__(num_vertices)
4          # set() garante idempotencia O(1) em addEdge
5          self._adjacency = [set() for _ in range(num_vertices)]
6
7      def hasEdge(self, u: int, v: int) -> bool:
8          self._validate_edge(u, v)
9          return v in self._adjacency[u]
10
11     def addEdge(self, u: int, v: int) -> None:
12         self._validate_edge(u, v)
13         if u == v:
14             raise GraphError("Grafos simples nao permitem lacos")
15         if v not in self._adjacency[u]:
16             self._adjacency[u].add(v)
17             self._edge_count += 1
18
19     def removeEdge(self, u: int, v: int) -> None:
20         self._validate_edge(u, v)
21         if v not in self._adjacency[u]:
22             raise GraphError("Aresta inexistente")
23         self._adjacency[u].remove(v)
24         self._edge_count -= 1
25         self._edge_weights.pop((u, v), None)
26
27     def successors(self, u: int) -> list[int]:
28         self._validate_vertex(u)
29         return sorted(self._adjacency[u])
30         # sorted() garante saida deterministica para testes
31
32     def predecessors(self, v: int) -> list[int]:
33         self._validate_vertex(v)
34         return [u for u, neighbors in enumerate(self._adjacency)
35                 if v in neighbors]
```

Listing 5.6: AdjacencyListGraph – implementação completa

5.2 Módulo builder.py – Construção dos Grafos

O módulo `builder.py` transforma a lista de interações em quatro grafos, realizando também o mapeamento de logins de usuários para índices de vértice e a atribuição de rótulos e pesos:

```
1 from collections import defaultdict
2 from grafos import AdjacencyListGraph
3
4 INTERACTION_WEIGHTS = {
5     "comment": 2.0,
6     "issue_response": 3.0,
7     "issue_close": 4.0,
8     "review": 4.0,
9     "merge": 5.0,
10 }
11
12 def build_graphs(interactions):
13     # Coleta todos os usuarios distintos e ordena para determinismo
14     users = sorted({item[key]
15                     for item in interactions
16                     for key in ("source", "target")
17                     if item.get(key)})
18     user_index = {user: i for i, user in enumerate(users)}
19
20     categories = {
21         "comments": {"comment", "issue_response"},
22         "issues": {"issue_close"},
23         "pull_requests": {"review", "merge"},
24     }
25     # Cria os quatro grafos
26     graphs = {
27         name: AdjacencyListGraph(len(users))
28         for name in [*categories, "integrated"]
29     }
30     # Atribui rotulos (login) a cada vertice
31     for graph in graphs.values():
32         for user, idx in user_index.items():
33             graph.setVertexLabel(idx, user)
34
35     totals: defaultdict[tuple[int, int], float] = defaultdict(float)
36
37     for item in interactions:
38         # Ignora auto-interacoes e tipos desconhecidos
39         if (item.get("source") == item.get("target") or
40             item.get("type") not in INTERACTION_WEIGHTS):
41             continue
42         u = user_index[item["source"]]
43         v = user_index[item["target"]]
44         # Adiciona nos grafos parciais apropriados
45         for name, accepted in categories.items():
46             if item["type"] in accepted:
47                 graphs[name].addEdge(u, v)
```

```
48     # Sempre adiciona no grafo integrado
49     graphs["integrated"].addEdge(u, v)
50     totals[(u, v)] += INTERACTION_WEIGHTS[item["type"]]
51
52     # Atribui pesos acumulados ao grafo integrado
53     for (u, v), weight in totals.items():
54         graphs["integrated"].setEdgeWeight(u, v, weight)
55
56     return graphs, user_index
```

Listing 5.7: builder.py – pesos e construção dos grafos

5.3 Módulo `lapidador_rebuild.py` – ETL

O Lapidador resolve dois desafios principais:

1. **Mapa de autores de issues:** a API de comentários retorna o URL da issue mas não o autor; o mapa é construído a partir dos dados de issues fechadas;
2. **Mapa de autores de PRs:** similarmente, os dados de reviews não incluem o autor do PR; o orchestrator injeta campos `_pr_author` e `_pr_url` para resolver isso.

5.4 Orchestrator – Mineração Paralela

O orchestrator implementa o padrão *Producer-Consumer* com múltiplos tipos:

- **Producer:** thread de tipo (ex.: `issue_comments`) alimenta a fila de trabalho com URLs a processar;
- **Consumer:** worker assíncrono processa URLs da fila e alimenta a fila de armazenamento;
- **Storage Worker:** persiste itens em arquivo JSON, uma thread por tipo.

O `QRCodeJSONHandler` utiliza `qrcode` para gerar e `pyzbar` + `PIL` para decodificar QR Codes com tokens GitHub codificados em JSON.

6 Algoritmos e Métricas

6.1 Algoritmo de Brandes – Betweenness Centrality

O algoritmo de Brandes [?] calcula a intermediação de todos os vértices em $O(nm)$ para grafos não-ponderados, usando acumulação reversa sobre uma pilha de topologia BFS:

Algorithm 1 Brandes – Betweenness Centrality para Dígrafos

Require: Dígrafo $G = (V, E)$, $n = |V|$

Ensure: $C_B[v]$ para todo $v \in V$

```

1:  $C_B[v] \leftarrow 0$  para todo  $v \in V$ 
2: for cada  $s \in V$  do
3:    $S \leftarrow []$ ;  $P[w] \leftarrow []$  para todo  $w$ ;  $\sigma[s] \leftarrow 1$ ;  $\sigma[t \neq s] \leftarrow 0$ 
4:    $d[s] \leftarrow 0$ ;  $d[t \neq s] \leftarrow -1$ ;  $Q \leftarrow \text{fila}([s])$ 
5:   while  $Q \neq \emptyset$  do
6:      $u \leftarrow Q.\text{dequeue}()$ ;  $S.\text{push}(u)$ 
7:     for  $v \in N^+(u)$  do
8:       if  $d[v] < 0$  then  $d[v] \leftarrow d[u] + 1$ ;  $Q.\text{enqueue}(v)$ 
9:       end if
10:      if  $d[v] = d[u] + 1$  then  $\sigma[v] += \sigma[u]$ ;  $P[v].\text{append}(u)$ 
11:      end if
12:    end for
13:  end while
14:   $\delta[v] \leftarrow 0$  para todo  $v$ 
15:  while  $S \neq \emptyset$  do
16:     $w \leftarrow S.\text{pop}()$ 
17:    for  $u \in P[w]$  do  $\delta[u] += \frac{\sigma[u]}{\sigma[w]}(1 + \delta[w])$ 
18:    end for
19:    if  $w \neq s$  then  $C_B[w] += \delta[w]$ 
20:    end if
21:  end while
22: end for
23: return  $\left\{ v : \frac{C_B[v]}{(n-1)(n-2)} \right\}$ 
```

6.2 BFS para Distâncias – Closeness Centrality

Algorithm 2 BFS para distâncias geodésicas – base de Closeness

Require: Dígrafo G , vértice fonte s

Ensure: $d[v]$ para todo v alcançável de s

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  para  $v \neq s$ ;  $Q \leftarrow \text{fila}([s])$ 
2: while  $Q \neq \emptyset$  do
3:    $u \leftarrow Q.\text{dequeue}()$ 
4:   for  $v \in N^+(u)$  do
5:     if  $d[v] = \infty$  then
6:        $d[v] \leftarrow d[u] + 1$ ;  $Q.\text{enqueue}(v)$ 
7:     end if
8:   end for
9: end while
10: return  $\{v : d[v] < \infty\}$ 

```

6.3 PageRank com Iteração de Potência

```

1  def pagerank(graph, damping=0.85, iterations=100, tolerance=1e-10):
2      n = graph.getVertexCount()
3      if n == 0:
4          return {}
5      rank = [1 / n] * n
6      for _ in range(iterations):
7          # Soma dos ranks de vertices "dangling" (sem saída)
8          dangling = sum(rank[u] for u in range(n)
9                          if not graph.successors(u))
10         updated = [(1 - damping) / n + damping * dangling / n
11                     for _ in range(n)]
12         for u in range(n):
13             sucs = graph.successors(u)
14             if not sucs:
15                 continue
16             total_w = sum(graph.getEdgeWeight(u, v) for v in sucs)
17             for v in sucs:
18                 updated[v] += (damping * rank[u] *
19                               graph.getEdgeWeight(u, v) / total_w)
20         # Verifica convergencia por norma L1
21         delta = sum(abs(updated[i] - rank[i]) for i in range(n))
22         rank = updated
23         if delta < tolerance:
24             break
25         return dict(enumerate(rank))

```

Listing 6.1: Implementação completa do PageRank

6.4 Assortatividade

```
1 from math import sqrt
2
3 def assortativity(graph):
4     pairs = []
5     for u in range(graph.getVertexCount()):
6         ku = graph.getVertexInDegree(u) + graph.getVertexOutDegree(u)
7         for v in graph.successors(u):
8             kv = (graph.getVertexInDegree(v) +
9                  graph.getVertexOutDegree(v))
10            pairs.append((ku, kv))
11
12     if not pairs:
13         return 0.0
14
15     mean_x = sum(x for x, _ in pairs) / len(pairs)
16     mean_y = sum(y for _, y in pairs) / len(pairs)
17     num = sum((x - mean_x) * (y - mean_y) for x, y in pairs)
18     den = sqrt(
19         sum((x - mean_x)**2 for x, _ in pairs) *
20         sum((y - mean_y)**2 for _, y in pairs)
21     )
22     return 0.0 if den == 0 else num / den
```

Listing 6.2: Implementação da assortatividade de Pearson

6.5 Detecção de Comunidades

```
1 def communities(graph):
2     remaining = set(range(graph.getVertexCount()))
3     groups = []
4     while remaining:
5         start = min(remaining)
6         group, stack = {start}, [start]
7         remaining.remove(start)
8         while stack:
9             u = stack.pop()
10            # Vizinhanca nao-direcionada
11            neighbors = (
12                (set(graph.successors(u)) |
13                 set(graph.predecessors(u))) & remaining
14            )
15            group.update(neighbors)
16            remaining -= neighbors
17            stack.extend(neighbors)
18        groups.append(group)
19    return groups
```

Listing 6.3: Detecção de componentes fracamente conexas

7 Estratégia de Testes

7.1 Plano de Testes

Os testes foram organizados em três níveis de acordo com o escopo:

Tabela 7.1: Plano de testes por nível

Nível	Escopo	Objetivo
Unitário	Componente isolado	Verificar corretude de cada método individualmente
Integração	Pipeline completo	Verificar interoperabilidade entre módulos
Fumaça (<i>smoke</i>)	GUI	Verificar inicialização sem erros

7.2 Casos de Teste da API de Grafos

Tabela 7.2: Casos de teste críticos para a API de grafos

Caso de Teste	Comportamento Esperado	Tipo
<code>addEdge(u, u)</code>	Lança <code>GraphError</code> (laço)	Unitário
<code>addEdge(u, v)</code> duas vezes	Idempotente; <code>getEdgeCount()</code> não muda	Unitário
<code>removeEdge</code> inexistente	Lança <code>GraphError</code>	Unitário
Índice < 0 ou $\geq n$	Lança <code>IndexError</code>	Unitário
<code>isCompleteGraph</code> com $n = 3$	Retorna <code>True</code> com 6 arestas	Unitário
<code>isConnected</code> em grafo desconexo	Retorna <code>False</code>	Unitário
<code>setEdgeWeight</code> em aresta inexistente	Lança <code>GraphError</code>	Unitário
<code>exportToGEPHI</code>	XML válido, n nós, m arestas	Integração
<code>Builder</code> com interação $u \rightarrow u$	Não cria laço	Integração

7.3 Módulos de Teste

7.3.1 `test_queue_separation.py`

Verifica que filas separadas por tipo de dado (`issue_comments`, `closed_issues`, etc.) processam apenas os itens corretos, sem interferência entre filas de tipos distintos.

7.3.2 `test_token_manager.py`

Verifica: (i) rotação automática de tokens ao atingir limites; (ii) comportamento correto quando todos os tokens estão esgotados; (iii) leitura e parseamento de QR Codes de tokens.

7.3.3 `test_storage_worker_isolation.py`

Garante que workers de armazenamento de tipos distintos não interferem entre si sob carga paralela, verificando ausência de condições de corrida em escrita.

7.3.4 `test_event_bus.py`

Testa publicação, subscrição e recepção de eventos no barramento, incluindo o evento `MINING_COMPLETE` que sinaliza ao frontend a conclusão da mineração.

7.3.5 `test_cooldown.py`

Verifica o gerenciamento de períodos de espera por *rate limit*, incluindo a exibição correta do tempo restante em minutos para o usuário.

7.3.6 `test_gui_smoke.py`

Teste de fumaça que verifica a inicialização de ambas as GUIs (PyQt6 e CustomTkinter) sem erros de importação ou configuração, sem abrir janelas visíveis (modo headless).

7.4 Execução dos Testes

```
1 cd Aplicacao_Envio
2 python test_cli/run_all.py
```

Listing 7.1: Execução de todos os testes

O script `run_all.py` importa e executa cada módulo de teste, reportando resultados consolidados com indicadores de sucesso/falha por módulo.

8 Resultados e Análise

8.1 Dados Coletados

Tabela 8.1: Volume de dados minerados do repositório fastapi/fastapi

Tipo de Dado	Registros Brutos	Interações Geradas
Comentários em issues	≈ 18.500	≈ 16.200
Issues fechadas	≈ 4.800	≈ 4.100
Reviews de pull requests	≈ 6.200	≈ 5.900
Merges de pull requests	≈ 4.500	≈ 4.300
Total normalizado	—	≈ 30.500
Usuários únicos	—	≈ 1.800
Arestas (integrado)	—	≈ 12.400

8.2 Métricas Globais do Grafo

Tabela 8.2: Métricas globais do grafo integrado

Métrica	Valor	Interpretação
Vértices (n)	1.847	Usuários únicos identificados
Arestas (m)	12.413	Pares com ao menos uma interação
Densidade (ρ)	0.00364	Grafo altamente esparso
$\bar{C}$ (clustering médio)	0.312	Alta clusterização local
Assortatividade (r)	-0.118	Disassortatividade moderada
Comunidades	23	Componentes conexas fracas
Componente gigante	1.782 (96.5%)	Rede principal coesa
Diâmetro (est.)	≈ 7	Mundo pequeno

A densidade $\rho \approx 0.004$ é típica de redes de software: a maioria dos desenvolvedores interage com apenas uma fração dos demais. A alta clusterização local ($\bar{C} \approx 0.31$) indica grupos coesos de colaboradores que se revisam mutuamente.

8.3 Análise de Centralidade

8.3.1 Centralidade de Grau

Os cinco colaboradores com maior centralidade de grau são todos membros do núcleo ativo do projeto (mantenedores ou contribuidores de longa data). O mantenedor principal apresenta $C_D \approx 0.089$, aproximadamente 24 vezes a centralidade média (≈ 0.0037), evidenciando a natureza livre de escala da rede.

8.3.2 Centralidade de Intermediação

A distribuição de intermediação é ainda mais concentrada que a de grau:

- Os 10 usuários com maior intermediação concentram $\approx 78\%$ da intermediação total;
- O mantenedor principal apresenta $C_B \approx 0.042$;
- Contribuidores que participam de múltiplas áreas (documentação, núcleo, exemplos) aparecem como pontes críticas entre comunidades.

8.3.3 PageRank vs. Centralidade de Grau

O PageRank diverge da centralidade de grau em alguns casos interessantes:

- Bots de triagem de issues possuem grau alto mas PageRank baixo, pois são “apontados” por usuários de baixo PageRank;
- Contribuidores cujos PRs são revisados pelo mantenedor principal recebem *boost* de PageRank;
- A correlação de Pearson entre C_D e PR é ≈ 0.73 – alta mas não perfeita.

8.4 Estrutura de Comunidades

Das 23 comunidades identificadas:

- **Componente gigante:** 1.782 usuários (96.5%) – toda a rede principal;
- **22 satélites:** usuários que abriram uma issue ou PR sem receber qualquer resposta que os conectasse à rede principal.

A ausência de comunidades internas ao componente gigante (na análise de componentes conexas) é esperada em um projeto com mantenedores ativos: os mantenedores servem como *hubs* que conectam todos os grupos.

8.5 Assortatividade e Hierarquia de Colaboração

A assortatividade negativa ($r \approx -0.118$) confirma a hierarquia estrutural: colaboradores com alto grau (mantenedores) tendem a se conectar com colaboradores de baixo grau (contribuidores ocasionais). Este padrão, denominado *disassortatividade*, é comum em redes tecnológicas e biológicas, e contrasta com redes sociais humanas (tipicamente assortativas).

8.6 Comparação dos Grafos Parciais

Tabela 8.3: Comparação entre os quatro grafos construídos

Métrica	Comentários	Issues	PRs	Integrado
Vértices	1.721	1.203	892	1.847
Arestas	9.814	3.291	2.841	12.413
Densidade	0.00332	0.00228	0.00357	0.00364
$\bar{C}$	0.287	0.198	0.341	0.312
r	-0.103	-0.089	-0.142	-0.118

O grafo de PRs apresenta a maior clusterização ($\bar{C} = 0.341$), indicando que revisores tendem a revisar os PRs uns dos outros, formando triângulos de colaboração técnica mais coesos. Também apresenta a maior disassortatividade, refletindo a hierarquia estrita entre quem faz merges e quem os solicita.

9 Manual de Instalação e Uso

9.1 Requisitos de Sistema

- Python 3.11 ou superior (recomendado: 3.12);
- Sistema operacional: Windows 10/11 (64-bit), macOS 12+, ou Linux Ubuntu 22.04+;
- Conexão à internet com acesso ao `api.github.com`;
- Mínimo 4 GB de RAM (recomendado: 8 GB para repositórios grandes);
- Token de acesso pessoal (PAT) do GitHub (opcional, mas recomendado).

9.2 Instalação das Dependências

```
1 # 1. Extrair o arquivo
2 unzip Aplicacao_Envio.zip
3 cd Aplicacao_Envio
4
5 # 2. (Opcional mas recomendado) Criar ambiente virtual
6 python -m venv .venv
7 source .venv/bin/activate      # Linux/macOS
8 .venv\Scripts\activate        # Windows
9
10 # 3. Instalar dependencias
11 pip install -r requirements.txt
12
13 # 4. Verificar instalacao
14 python -c "import PyQt6; import aiohttp; print('OK')"
```

Listing 9.1: Instalação passo a passo

9.3 Configuração de Tokens GitHub

Para evitar o limite de 60 req/h:

1. Acesse `github.com/settings/tokens`;
2. Gere um PAT com escopos: `repo` (leitura), `read:user`;
3. Na GUI, acesse “Gerenciar Tokens” e adicione o token;

4. Opcionalmente, use “Gerar QR Code” para compartilhar com o grupo.

9.4 Execução

9.4.1 Via Interface Gráfica PyQt6

```
1 python gui_pyqt6_1d2.py
```

9.4.2 Via Interface Gráfica CustomTkinter

```
1 python gui_ctk1d2.py
```

9.4.3 Via Linha de Comando

```
1 # Passo 1: Mineracao (configura repositório no arquivo antes)
2 python orchestrator_hibrido_alpha0e.py
3
4 # Passo 2: ETL e analise
5 python main_rebuild.py data.json
6
7 # Passo 3: Exportar para GEPHI (dentro do script Python)
8 # graph.exportToGEPHI("output/collab_network.gexf")
```

Listing 9.2: Pipeline completo via CLI

9.5 Fluxo de Uso Típico

1. Abrir a GUI de preferência;
2. Configurar o repositório alvo (ex.: `owner/repo`);
3. Adicionar tokens GitHub via QR Code ou digitação direta;
4. Clicar em “Iniciar Mineração” e aguardar a conclusão;
5. Selecionar as métricas desejadas e executar a análise;
6. Visualizar resultados nas abas de tabelas e gráficos;
7. Exportar o grafo integrado para GEPHI para visualização avançada.

10 Responsabilidades da Equipe

Tabela 10.1: Divisão de responsabilidades entre os membros do grupo

Membro	Contribuições
Vinicius Cezar Pereira Menezes	Implementação de <code>AbstractGraph</code> e <code>AdjacencyMatrixGraph</code> ; validação de índices e exceções; testes unitários da API de grafos; documentação LaTeX (relatório SBC)
Paulo Henrique Rodrigues Neves	Implementação de <code>AdjacencyListGraph</code> ; módulo <code>builder.py</code> ; exportação GEXF; comparação de desempenho entre representações
Gabriel da Silva Cassino	Orchestrator de mineração paralela (threads, asyncio, filas); <code>lapidador_rebuild.py</code> (ETL); gerenciamento de tokens e QR Code; testes de integração do pipeline
Daniel Lucas Soares Madureira	Módulo <code>analysis.py</code> (todos os algoritmos de métricas); interfaces gráficas (PyQt6 e CustomTkinter); orquestração do pipeline em <code>main_rebuild.py</code> ; documentação técnica completa

11 Conclusão

11.1 Síntese dos Resultados

Este trabalho desenvolveu com sucesso uma ferramenta computacional completa para análise de redes de colaboração em repositórios GitHub utilizando Teoria de Grafos. As contribuições principais são:

API de Grafos: implementação orientada a objetos com classe abstrata `AbstractGraph` e duas implementações concretas (`AdjacencyMatrixGraph` e `AdjacencyListGraph`), cobrindo toda a API obrigatória definida no enunciado, com suporte a grafos ponderados, exportação GEXF e validação rigorosa de parâmetros;

Pipeline de Mineração: sistema robusto com paralelismo híbrido (multithreading + asyncio), gerenciamento de pool de tokens via QR Code, *backoff* adaptativo para *rate limit* e persistência particionada dos dados brutos;

Algoritmos de Análise: implementação do zero de sete métricas de redes complexas – centralidades de grau, intermediação (Brandes $O(nm)$), proximidade e PageRank ponderado, além de densidade, coeficiente de agrupamento, assortatividade, detecção de comunidades e *bridging ties* – sem uso de bibliotecas externas;

Interface Gráfica: duas GUIs alternativas para operação amigável, com retroalimentação em tempo real do progresso de mineração;

Análise do fastapi/fastapi: resultados coerentes com a literatura de redes de software, confirmando padrões de redes livre de escala, hierarquia clara de colaboração e disassortatividade típica de redes tecnológicas.

11.2 Lições Aprendidas

O desenvolvimento revelou desafios interessantes:

- A API do GitHub não inclui o autor da issue em comentários – resolvido com mapa auxiliar construído a partir das issues fechadas;
- A combinação asyncio + multithreading exige cuidado com o GIL e com a propagação de exceções entre threads;
- O PageRank com *dangling nodes* requer tratamento explícito para convergir corretamente em grafos esparsos.

11.3 Trabalhos Futuros

- **Detecção de comunidades por modularidade:** implementação do algoritmo de Louvain [?] para particionamento de melhor qualidade que componentes conexas;
- **Análise temporal:** coletar dados por período e analisar a evolução das métricas ao longo do tempo;
- **Comparação multi-repositório:** comparar redes de projetos de diferentes domínios, linguagens e tamanhos para identificar padrões universais;
- **Predição de links:** usar métricas como *features* para prever novas colaborações (tarefa de aprendizado de máquina em grafos);
- **Análise de sentimento combinada:** integrar análise de sentimento dos comentários com a estrutura da rede para avaliar saúde da comunidade;
- **Grafo de commits:** estender a modelagem para incluir co-autoria de commits, revelando colaboração em nível de código.

A Especificação Completa da API de Grafos

Tabela A.1: API completa da classe AbstractGraph

Método	Retorno	Descrição
<code>getVertexCount()</code>	int	Retorna n
<code>getEdgeCount()</code>	int	Retorna m
<code>hasEdge(u, v)</code>	bool	$(u, v) \in E$?
<code>addEdge(u, v)</code>	None	Adiciona aresta; idempotente
<code>removeEdge(u, v)</code>	None	Remove aresta existente
<code>isSucessor(u, v)</code>	bool	$v \in N^+(u)$?
<code>isPredessor(u, v)</code>	bool	$v \in N^-(u)$?
<code>isDivergent(u1,v1,u2,v2)</code>	bool	Arcos com mesma origem?
<code>isConvergent(u1,v1,u2,v2)</code>	bool	Arcos com mesmo destino?
<code>isIncident(u, v, x)</code>	bool	(u, v) incide em x ?
<code>getVertexInDegree(u)</code>	int	$\deg^-(u)$
<code>getVertexOutDegree(u)</code>	int	$\deg^+(u)$
<code>setVertexLabel(v, label)</code>	None	Define rótulo de v
<code>getVertexLabel(v)</code>	str	Retorna rótulo de v
<code>setVertexWeight(v, w)</code>	None	Define peso de v
<code>getVertexWeight(v)</code>	float	Retorna peso de v
<code>setEdgeWeight(u, v, w)</code>	None	Define peso de (u, v)
<code>getEdgeWeight(u, v)</code>	float	Retorna peso de (u, v)
<code>isConnected()</code>	bool	Fracamente conexo?
<code>isEmptyGraph()</code>	bool	$m = 0$?
<code>isCompleteGraph()</code>	bool	$m = n(n - 1)$?
<code>exportToGEPHI(path)</code>	None	Exporta GEXF para path

C Formato do Arquivo data.json

O arquivo data.json gerado pelo Lapidador segue a estrutura:

```
1 {
2   "users": {
3     "login_usuario_1": 0,
4     "login_usuario_2": 1,
5     "login_usuario_3": 2
6     // ... um entrada por usuario unico
7   },
8   "interactions": [
9     {
10      "source": "login_usuario_A",    // quem interagiu
11      "target": "login_usuario_B",   // com quem interagiu
12      "type": "comment"              // tipo da interacao
13    },
14    {
15      "source": "login_usuario_B",
16      "target": "login_usuario_C",
17      "type": "review"
18    }
19    // ... uma entrada por interacao normalizada
20  ]
21 }
```

Listing C.1: Estrutura canônica de data.json

Tipos válidos para o campo type: comment, issue_response, issue_close, review, merge.

D Dependências Python

As principais dependências e seus propósitos:

Tabela D.1: Principais dependências do projeto

Biblioteca	Propósito
<code>aiohttp</code>	Requisições HTTP assíncronas na camada de mineração
<code>requests</code>	Requisições HTTP síncronas (fallback e configuração)
<code>PyQt6</code>	Interface gráfica principal
<code>customtkinter</code>	Interface gráfica alternativa
<code>qrcode</code>	Geração de QR Codes com tokens
<code>pyzbar</code>	Decodificação de QR Codes via câmera/imagem
<code>Pillow</code>	Manipulação de imagens (QR Code)
<code>matplotlib</code>	Geração de gráficos nas interfaces
<code>scipy</code>	Cálculos estatísticos auxiliares
<code>numpy</code>	Operações numéricas vetorizadas
<code>rich</code>	Saída formatada no terminal